

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[module-4.2]

DISTRIBUTED ALGORITHMS - AN OVERVIEW

TEXTBOOK REFERENCE

- These slides are based on the textbooks:
 - “Elements of Distributed Computing”, Vijay K. Garg
 - “Distributed Systems - Concepts and Design” Coulouris et al.

DISTRIBUTED ALGORITHMS

- Main challenges
 - no shared clock, no shared memory, failures
- General idea
 - using logical clocks and *happened before* relation to define an order between (related) events
 - assumption for most algorithms:
 - *asynchronous systems*, but no failures
 - finite but not known a priori delivery time
 - *synchronous systems* when dealing with failures
 - timeouts

MAIN CATEGORIES

- **Events ordering (partial)**
 - Logical clocks, vector clocks
 - discussed in previous module
 - used by all other algorithms
- **States Observation**
 - Global snapshots
- **Coordination & Agreement**
 - Distributed Mutual Exclusion
 - Election
 - Multicast communication
 - Consensus
- **Distributed Transaction and Concurrency Control**
- **Replication & Fault-Tolerant Services**

IN THIS MODULE

- **Mutual exclusion**
 - centralised algorithm
 - Ricart-Agrawala decentralised algorithm
- **Leader election**
 - Chang-Roberts algorithm
- **Message ordering**
 - causal message ordering
 - total message ordering
- **Consensus**
 - consensus problem definition
- **Global snapshots**
 - Chandy-Lamport algorithm

MUTUAL EXCLUSION

MUTUAL EXCLUSION & CRITICAL SECTION

- Analogous to the problem in the concurrent case
 - here processes are distributed
- Critical section properties
 - **safety**
 - two processes should not have the permission to be in their CS simultaneously
 - **liveness**
 - every request for the CS is eventually granted
 - **fairness**
 - Different requests must be granted in the order they are made

CENTRALIZED ALGORITHM

- There is a *coordinator* giving the token
- Key point: **fairness**
 - requests should be given access according the happened before relation
 - given s, t = two local states (of different processes), if $s \rightarrow t$ then request done in s should be served before request done in t
 - even if a request done in t arrives to the coordinator before a request done in s , the coordinator must understand that there is a pending request from the process P related to s
 - the coordinator can delay the receiving of the message t until all requests before t are received
 - when communicating with the coordinator, a process includes its knowledge about the requests done so (far by all processes), so that if $s \rightarrow t$, then t has the information about s
 - information about requests known at the state s are captured using a vector $s.v$, where $s.v[j]$ represents the number of requests made by P_j that causally precede the state s

CENTRALIZED ALGORITHM

- Idea
 - processes cooperate by piggybacking vector clock v on all outgoing msgs
 - the coordinator process delays a request t until all requests before t are received
- Data structures on the coordinator side P_0
 - `reqlist`
 - list of requests received that have not been fulfilled yet
 - a request w is **eligible** in this list *if there is no request that happened before w and has not been fulfilled yet*
 - `reqdone`
 - vector such that `reqdone[i]` = number of requests made by P_i that have been fulfilled

Client process P_i

```
P[i]::  
var  
  v: array[1..N] of integer := [0]  
  inCS: boolean := false  
  
To request:  
  v[i] := v[i] + 1  
  send (req, v) to P[0]  
  
Upon receive(token) from P[0]:  
  inCS := true  
  
To release:  
  send token to P[0]  
  inCS := false  
  
Upon receive(u):  
  v := max(v, u.v)
```

A request w is *eligible* if $w.v$ is at most reqdone, i.e. there is no request that happened before w and has not been fulfilled yet

Coordinator

```
P[0]::  
var  
  reqDone: array[1..N] of integer := [0]  
  reqList: list of (pid, reqvector) := empty  
  haveToken: boolean := true  
  
Upon receive(u, req):  
  append(reqList, u)  
  if (haveToken)  
    checkReq()  
  
Upon receive(u, token):  
  haveToken := true  
  checkReq()  
  
checkReq:  
  eligible := {w in reqList |  
    for each j, if j != w.p: w.v[j] == reqDone[j] }  
    if j == w.p: w.v[j] == reqDone[j] + 1 }  
  if (eligible != {})  
    w := first(eligible)  
    delete(reqList, w)  
    reqDone[w.p] := reqDone[w.p] + 1  
    send token to P[w.p]  
    haveToken := false
```

EXPLANATION

- Client process P_i
 - To request the token, the client simply implements its component in v vector and sends the request with its vector v to the coordinator P_0
 - On receiving the token, the client proc. can enter the critical section
 - To release the critical section, it simply returns the token to P_0
 - To ensure fairness, processes cooperate by piggybacking v on all outgoing messages
 - When a process receives a message, it updates its v vector with component-wise max of its old value and the vector received with the msg
- Coordinator coordinator P_0
 - It maintains a vector *reqdone* with the interpretation that *reqdone[i]* equals the number of requests made by P_i that have been fulfilled
 - It also maintains *reqlist*, which is the list of requests received by P_0 that have not fulfilled yet
 - On receiving a request
 - P_0 appends it to *reqlist*
 - If it has the token, then it checks for **eligible** requests and sends the token to one of the eligible processes
 - A request w is *eligible* if $w.v$ is at most redone, that is: no requests that happened before w and had not been fulfilled yet

DECENTRALIZED SOLUTION: RICART AGRAWALA'S ALGORITHM

- Idea:
 - to request a resource, a process sends a timestamped message to all processes
 - on receiving a request from any other process, a process sends an OK message if either the process is not interested in entering in CS or if its own request has a higher timestamp value. Otherwise that process is kept in a pending queue
 - To release a resource, a process sends OK to all the processes in the pending queue
 - A process is granted the resource when it has requested the resource and it has received the OK message from every other process in response to its request msg

G.Ricart, A.K. Agrawala. *An optimal algorithm for mutual exclusion in computer networks.* Communication of ACM, 24, 1981

```

P[i]::

var
    pendingQ: list of process ids initially null
    myts: integer timestamp initially infinity
    numOK: integer initially 0

request:
    myts := logical_clock
    send req with myts to all other processes
    numOK := 0

receive(u, req):
    if (u.myts < myts)
        send OK to process P[u.pid]
    else
        append(pendingQ, u.pid)

receive(u, OK):
    numOK := numOK + 1
    if (numOK == N-1)
        enter_CS

release:
    myts := infinity
    for each pid in pendingQ
        send OK to P[pid]
    empty(pendingQ)

```

- Number of messages: $2(N-1)$
- It works with channels not necessarily FIFO

LEADER ELECTION

LEADER ELECTION

- Choosing one process among a set of N processes
- Based on superimposing a *logical ring topology* on the underlying network to execute control function
- Widely used strategy to choose coordinators in centralized algorithms

CHANG-ROBERTS ALGORITHM

- Assumption
 - each process has a unique Pid
- Idea
 - the process with the max Pid is elected
 - one or more processes can spontaneously wake up and initiate an election
 - a process
 - sends an election msg along with its identifier to its left, if it has not seen any msg with a higher identifier than its own
 - it forwards any message that has an identifier greater than its own, otherwise it swallows the message
 - if a process receives its own message => it declares itself as a leader and propagate a leader msg

E.J.H. Chang, R. Roberts. *An improved algorithm for decentralized extrema-finding in circular configurations of processes*. Communications of ACM, 22(5):281-283, 1979.


```

P[i]::

var
    myid: integer
    awake: boolean := false
    leaderId: integer := null

To initiate election:
    send (election,myid) to P[i-1]
    awake := true

Upon receiving a message (election,j):
    if (j > myid)
        send (election,j) to P[i-1]
    else if (j == myid)
        send (leader,myid) to P[i-1]
    else if ((j < myid) and !awake)
        send (election, myid) to P[i-1]
    awake := true

Upon receiving a message (leader,j):
    leaderId := j
    if (j != myid)
        send (leader,j) to P[i-1]

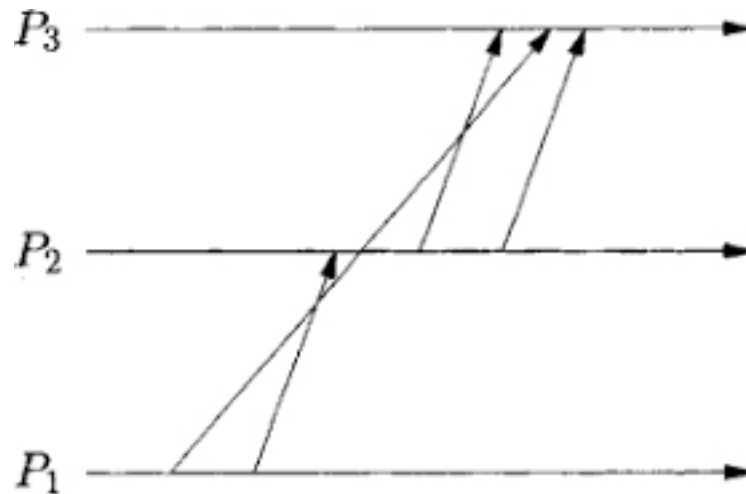
```

- Number of messages exchanged
 - worst case: election msgs are $2N-1$ election + N leader msgs

MESSAGE ORDERING

CAUSAL MESSAGE ORDERING

- Fully async computation => no restrictions in msg ordering
 - max non-determinism, max concurrency
- Sometimes it is useful to reduce non-determinism by restricting the possible message ordering in a system
- From FIFO-ordered computation to **causal ordering**
 - given 2 msgs m_1, m_2 such that $\text{send}(m_1) \rightarrow \text{send}(m_2)$, then $\text{rec}(m_1) \rightarrow \text{rec}(m_2)$
 - no overtaking



A FIFO computation that is not causally ordered
(CHAPTER 16, Garg)

CAUSAL MESSAGE ORDERING

- Algorithm ensuring causal ordering - Idea:
 - each process maintains a **matrix m** of integers (extension of vector clocks)
 - $s.m[j, k]$ = number of msgs sent by a process P_j to P_k as known by P_i in state s
 - whenever a msg is sent from P_i to P_j ,
 - the matrix m is piggybacked with the msg
 - $m[i, j]$ is incremented to reflect that one more msg has been sent from P_i to P_j
 - when a msg is received by the communicating system, first it is checked for *eligibility* before delivery to P_i
 - a msg u is **eligible** to be received in state s by P_i when the number of msgs sent from any process P_k to P_i as reported by $u.m$ is less or equal to the number recorded in $s.m$
 - if it is not eligible, the message is buffered until it can be received

```

Pi::

var
    /* local matrix clock m */
    m: array[1..N,1..N] of integer := 0

For every msg sent (from process Pi) to process Pj:
- m[i,j] := m[i,j] + 1
- piggyback the matrix m along with the msg

For every msg received (by Pi) from process Pj, with matrix m':

    the msg is eligible if
        -  $\forall k$ , if  $k \neq j$ :  $m'[k,i] == m[k,i]$ , and
          if  $k == j$ :  $m'[k,j] == m[k,j] + 1$ 

    if the message is not eligible
        then delay the msg (put the msg in a queue)
    else
        update m with m',
        receive the msg and
        check for other possible eligible messages

```

FURTHER EXPLORATIONS

- Synchronous / Total order
 - stronger than causal ordering
 - setting up a coordination such that all messages can be treated as logically instantaneous
- Total order for multicast messages
 - in the case that a message is sent to multiple processes, all messages are delivered in the same order at all processes
 - for all messages x and y and all processes P and Q , if x is received at process P before y , then y is not received before x at Q

SYNCHRONIZERS

- The design of distributed algorithms is easier *if we assume that the underlying network is synchronous rather than asynchronous*
 - methods to simulate sync network over async
 - transmission time ~ 0
 - assuming absence of failures or known max delay
- Mechanisms: **pulses** and **synchronizers**
 - a pulse is a counter at each process with the property that any msg sent in a pulse i is received at pulse i
 - a synchronizer is a mechanism that indicates to a process when it can generate a pulse
- Complexity
 - message complexity
 - additional number of messages required by the synchronizer to simulate a synch algorithm on top of an async network
 - time complexity
 - number of time units required to simulate one pulse where a time unit is defined as the time required for an async msg

SYNCHRONIZERS

- Idea
 - every process sends exactly one msg to all neighbors in each pulse
 - if a process is in round i and it gets a msg with pulse $i+1$, then it buffers that msg and waits for the msg with pulse number i

```
Pj::  
  
var  
    pulse: integer := 0  
  
round i:  
    pulse := pulse + 1  
    simulate the round i of the synch algorithm  
    send msg to all neighbors with pulse  
    wait for exactly one msg from each neighbor with (pulse = i)
```


GLOBAL STATE OBSERVATION

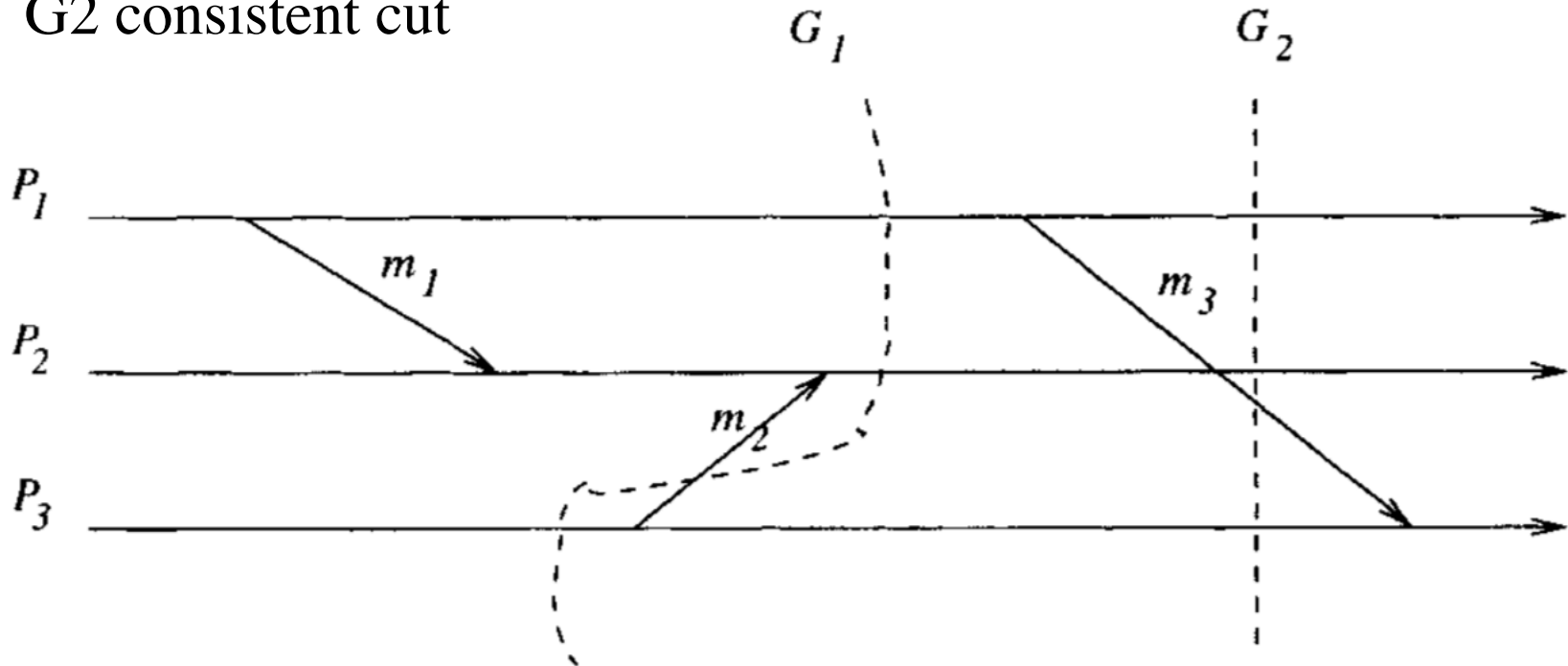
GLOBAL STATE

- Capturing the *global state* of a distributed system from a single process
- Def. **global state**
 - *is a set of local states that are all concurrent with each other (according to the happened before model)*
 - for many applications, it is enough to capture a global state that happened in the past, not the current
- An algorithm that captures a global state is called *global snapshot algorithm*
- Challenge
 - a *consistent* global state is *not* simply the product of local states
 - def. **CONSISTENT CUT**
 - a global state S where $\forall$ event $e \mid e = \text{receive}(\text{msg}),$ and $e' = \text{send}(\text{msg}), e' \rightarrow e \Rightarrow e' \in S$

CONSISTENT AND INCONSISTENT CUT

G1 inconsistent cut

G2 consistent cut



(Fig. from “Elements of Distributed Computing”, Vijay K. Garg)

CHANDY-LAMPORT SNAPSHOT ALGORITHM

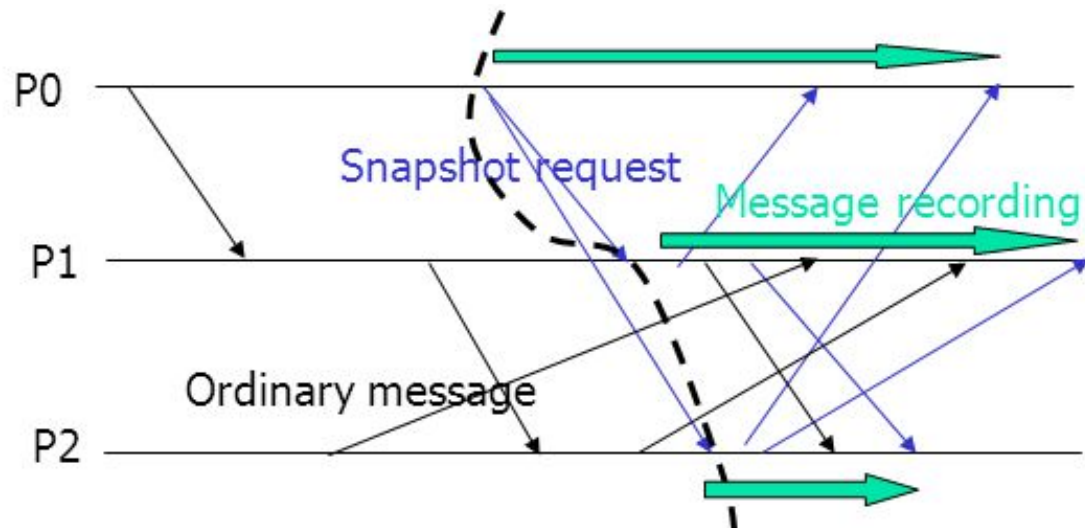
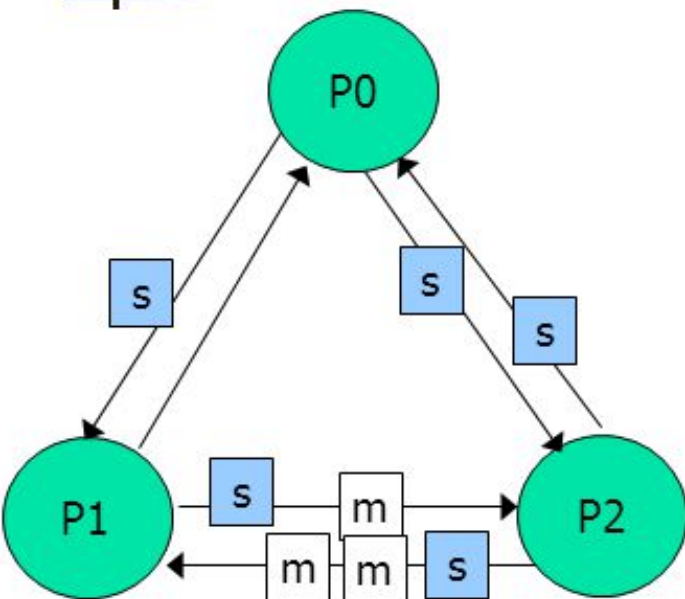
- Algorithm to take a global consistent snapshot of a distributed system, by computing a consistent cut
- Challenges
 - ensuring that local states are *concurrent*
 - doing the snapshot *without blocking* the processes
 - capturing the state of the channels
 - *marker* mechanism
- Assumptions
 - all channels are unidirectional and FIFO

C.K. Chandy, L. Lamport. *Distributed Snapshots: Determining global states of distributed systems*. ACM Transaction on Computer Systems. 3(1):63-75. Feb. 1985

CHANDY-LAMPORT SNAPSHOT ALGORITHM

- Idea
 - each process $\Rightarrow$ color variable (*red|white*): the computed global snapshot corresponds to the state of the system just before the processes turn to red
 - all processes are initially *white*
 - after recording a local state, a process turns to red
 - the state of a local process is simply the state just before it turned to red
 - To record the state of the channels:
 - once a process turns to red, it sends a marker msg along all its outgoing channels before it sends out any other msg
 - a process is required to turn red on receiving a marker if it has not already done so. Having FIFO channels:
 - no white process ever receives a message sent by a red process
 - this ensures that local states are mutually concurrent
 - *the state of the channels is given by messages sent by a white process and received by a red process*
 - msgs that were in transit when the snapshot is taken

CHANDY-LAMPORT SNAPSHOT ALGORITHM



- A process that wants to take a snapshot sends a snapshot request to the others.
- Each process records its state upon receiving the first snapshot request.
- Each process keep recording the messages until receiving a snapshot request from each of the other process except the one that has originally initiated a snapshot.

(From <http://slideplayer.com/slide/1506898/>)

```

Pi::

var
  color: {white,red} := white
  // assume k incoming channels
  chan: array[1..k] of queues of msg := empty // state of the channels to be recorded
  closed: array[1..k] of boolean := [false]

turn_red() enabled if (color = white):
  save_local_state
  color := red
  send maker to all neighbors

Upon receive(marker) on incoming channel j:
  if (color = white)
    turn_red()
  closed[j] := true

Upon receive(program_message) on incoming channel j:
  if (color = red and not closed[j])
    append(chan[j], program_message)

```

FURTHER ISSUES

- Observing *Global Predicates*
 - linear, regular, conjunctive, channel predicates
- Applications
 - deadlock detection
 - termination detection
- Computing *Global Functions*

CONSENSUS

CONSENSUS

- *Consensus* is a fundamental problem in distributed computing

finding agreement among distributed processes about the value of some property, about some action to do, etc.

- Consensus algorithms are pervasively used in real-world systems
 - transactions in DB (whether to commit a transaction), leader election (agreeing on the identity of a leader), state machine replication, atomic broadcasts, clock synchronization, blockchain, etc

CONSENSUS IN ASYNCHRONOUS NET

- In **asynchronous** network, in the presence of even a single unannounced process death/failure, the consensus problem is *impossible to solve*
 - **FLP result** (Fischer, Lynch, Patterson)

Given the general properties of consensus, all problems are impossible to solve in the case of process failure in the case of fully async networks..

Fischer, Lynch, Patterson. *Impossibility of distributed consensus with one faulty process.*
Journal of ACM, 32(2). April 1985

CONSENSUS UNDER SYNCHRONY

- Synchronous systems
 - an **upper bound** can be assumed on the message delay and on the duration of actions performed by processes
- Type of faults, models:
 - **crash**
 - crash model => a fault corresponds to a processor halting
 - **crash+link**
 - in this model either a processor can crash or a link may go down. If the link goes down, it stays down
 - **omission**
 - a process fails either by sending a proper subset of messages that it is required to send by the algorithm or by receiving only a proper subset of messages that have been sent to it
 - **byzantine failure**
 - a processor fails by exhibiting arbitrary behaviour

DEFINITION OF THE PROBLEM

- Definition of the problem
 - To reach consensus, every process P_i begins in the *undecided state* and proposes a single value v_i , drawn from a set D ($i = 1..N$)
 - The processes communicate with one another, exchanging values
 - Each process finally sets a *decision variable* d_i with the decided value
 - in doing so, it enters *the decided state*, in which it may no longer change d_i
- Requirements
 - **Termination.** Eventually each *correct* process sets its decision variable
 - **Agreement.** The decision value of all correct processes is the same: if P_i and P_j are correct and have entered the decided state, then $d_i = d_j$
 - **Integrity.** If the correct processes all proposed the same value, then any correct process in the *decided state* has chosen that value (i.e. don't change idea..)

BASIC ALGORITHM

- Idea
 - each process maintains V , the set of values it knows that have been proposed by processors in the system
 - initially a process knows only its own value
 - given the V set, it decides using some criteria / decision strategy
 - that should be the same for all processes
- Algorithm based on *rounds*
 - each correct process executes **$f + 1$ rounds**
 - where f = max number of processors that can fail
 - in each round
 - a process sends to all other processes values from V that it has not sent before
 - a process P_i receives the value sent by P_j
 - synchrony assumption: waiting some fixed predetermined time, after that it assumes that P_j crashed
- It works for in the case of no byzantine behaviours

Pi::

var

V: set of values := { v[i] }

for k := 1 to f + 1 do

send { v in V | Pi has not already sent v } to all

receive Sj from all process Pj, j != i

V := V U Sj

y := decide(V)

“*decide*” denotes the criteria (function) adopted to decide the value. The important thing here is that all processes would use the same criteria.

- Number of messages required: $O((f+1)N^2)$

CONSENSUS UNDER BYZANTINE FAULTS

- Case in which faulty processes can have arbitrary behaviour
 - sending wrong values
- **Byzantine General Agreement (BGA) problem**
 - designing a protocol for which the *loyal generals* (correct processes) can coordinate their actions (deciding to attack or retreat altogether) in spite of the treacherous processes (faulty)
- Result (theorem)
 - *There is no f -resilient protocol for BGA if $N \leq 3f$*
 - where N = total number of processes, f = number of faulty processes

L. Lamport, R. Shostak, M. Pease. *The Byzantine generals problem*. ACM Transaction on Programming Languages and Systems, 4(3):382-401, 1982

CONSENSUS UNDER BYZANTINE FAULTS

- Idea
 - $f+1$ rounds, f number of faulty processors
 - rotating coordinator (called **king**)
 - at least 1 round has a king which is not faulty
 - Processor P_i assumes to be the coordinator or king for round k
 - 2 stages
 - first stage: each processor exchanges its value to all other processors and based on V it determines its estimate
 - *myvalue* variable
 - second stage: it receives the value from the coordinator
 - if no value received (coordinator failed), then it assumes null
 - then it decides if keeping its own value or the king value
 - » the decision is based on the multiplicity of *myvalue* in the vector V
 - » if V has more than $N/2+f$ copies of *myvalue*, then *myvalue* is chosen for $V[i]$; otherwise kingvalue is used

```

Pi::

var
  V: array[1..N] of values := {v[i] = x, v[j] = not defined}

for k := 1 to f + 1 do
  first phase:
    send V[i] to all other processors
    set V[j] (j != i) to the value received from Pj
    myvalue := majority in the vector V (not defined, if no majority)

  second phase:
    if (k == i) // I'm the king in this round
      send myvalue to all other processors
      receive king value from Pk
    if V vector has more than N/2 + f copies of my values
      V[i] := myvalue
    else
      V[i] := kingvalue

y := V[i]

```

FURTHER EXPLORATIONS - PAXOS

- **Paxos**

- family of protocols for solving consensus in a network of *unreliable* processors
 - including a spectrum of trade-offs between the number of processors, number of message delays before learning the agreed value, the activity level of individual participants, number of messages sent, and types of failures
- used where durability is required
 - for example, to replicate a file or a database
- compared to the BGA problem, in this case we consider only crash-stop behaviour (not byzantine)
 - i.e. simpler case

Lamport, Leslie (May 1998). "*The Part-Time Parliament*". ACM Transactions on Computer Systems. 16 (2): 133–169.

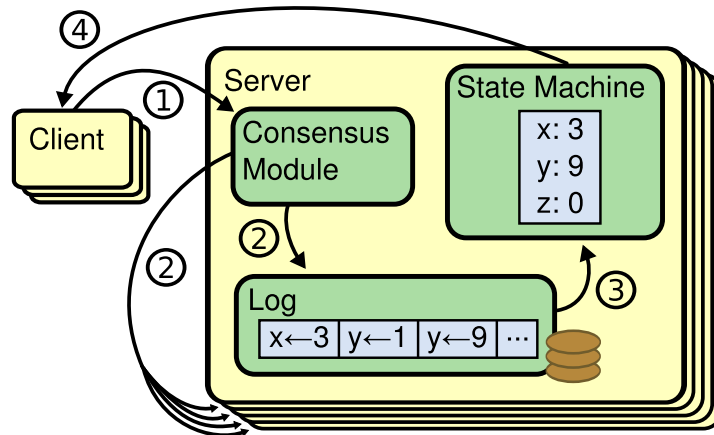
FURTHER EXPLORATIONS - RAFT

- **Raft (Reliable, Replicated, Redundant, And Fault-Tolerant)**
 - a consensus algorithm designed to be easy to understand
 - offers a generic way to distribute a state machine across a cluster of computing systems, ensuring that each node in the cluster agrees upon the same series of state transitions
 - <https://raft.github.io/>
 - it's equivalent to Paxos in fault-tolerance and performance, but easier to understand and implement
 - Open-source reference implementations available
 - Go, C++, Java, and Scala

Ongaro, Diego; Ousterhout, John (2013). "In Search of an Understandable Consensus Algorithm"

REPLICATED STATE MACHINES

- Consensus typically arises in the context of replicated state machines, a general approach to building fault-tolerant systems
- Each server has a state machine and a log
- The state machine is the component that we want to make fault-tolerant
 - e.g. a hash table



- It will appear to clients that they are interacting with a single, reliable state machine, even if a minority of the servers in the cluster fail.
- Each state machine takes as input commands from its log.
 - e.g. command: “set x to 3”

REPLICATED STATE MACHINES

- A consensus algorithm is used to agree on the commands in the servers' logs.
 - the consensus algorithm must ensure that if any state machine applies set x to 3 as the n th command, no other state machine will ever apply a different n th command.
- As a result, each state machine processes the same series of commands and thus produces the same series of results and arrives at the same series of states.